535510 Spring 2026: Reinforcement Learning **(Due: 2026/04/02, Thursday, 21:00)**

Homework 1: Fundamentals of MDPs and Policy Gradient

Submission Guidelines: Your deliverables shall consist of 2 separate files – (i) A PDF file: Please compile all your write-ups into one .pdf file (photos/scanned copies are acceptable; please make sure that the electronic files are of good quality and reader-friendly); (ii) A zip file: Please compress all your source code and demo video into one .zip file. Please submit your deliverables via E3.

Problem 1 (Regularized MDPs) (10+10=20 points)

In Lecture 4, we formally describe the regularized MDP, which is a direct extension of the classic MDP with a regularizer Ω. In this problem, for simplicity, suppose we use the Shannon entropy as our regularizer, i.e., $\Omega(\pi(\cdot|s)) \equiv H(\pi(\cdot|s)) := -\sum_{a\in\mathcal{A}} \pi(a|s) \ln \pi(a|s)$. Let us verify a few important properties mentioned in Lecture 4 as follows.

(a) Based on the regularized Bellman expectation equation, we can introduce the "*regularized Bellman expectation operator*" T^π_Ω as

$$[T^\pi_\Omega V](s) := R^\pi_s + \Omega(\pi(\cdot|s)) + \gamma P^\pi_{ss'} V. \tag{1}$$

Please verify that T^π_Ω is a contraction operator in ℓ_∞ norm. (Hint: Try to extend the proof of the contraction property of the unregularized operator T^π described in Lecture 3)

(b) Moreover, under regularized MDPs, we study the optimal value functions V^*_Ω and optimal Q functions Q^*_Ω and learn the Bellman optimality equations as

$$V^*_\Omega(s) = \max_{\pi\in\Pi} \{R^\pi_s + \gamma P^\pi_s V^*_\Omega\} \tag{2}$$

$$Q^*_\Omega(s,a) = R_{s,a} + \gamma E_{s'\sim P(\cdot|s,a)}[V^*_\Omega(s')]. \tag{3}$$

Could you design an iterative algorithm that can obtain V^*_Ω and Q^*_Ω? Please clearly write down the complete pseudo code of your algorithm and provide comments on each line of your pseudo code. (Hint: Try to extend the Value Iteration for standard MDPs to the regularized MDPs based on Equation (2))

Problem 2 (Policy Gradient) (8+4+8=20 points)

(a) Please prove the following useful property discussed in Lecture 4: For any function $f : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$,

$$\mathbb{E}_{\tau\sim P^{\pi_\theta}_\mu}\left[\sum_{t=0}^{\infty} \gamma^t f(s_t, a_t)\right] = \frac{1}{1-\gamma}\mathbb{E}_{s\sim d^{\pi_\theta}_\mu}\mathbb{E}_{a\sim\pi_\theta(\cdot|s)}\big[f(s,a)\big] \tag{4}$$

(Hint: It might be slightly easier to go from the RHS to LHS. Specifically, you may first expand the RHS of (4) into a sum of $f(s,a)$ over s and a and then apply the definition of $d^{\pi_\theta}_\mu$, which involves a sum of probability over t. Next, try to reorganize the triple summation into the form of the LHS of (4))

(b) Based on the above lemma and the REINFOCER policy gradient (i.e., (P2)), show the PG expression (P3):

$$\nabla_\theta V^{\pi_\theta}(\mu) = \frac{1}{1-\gamma}\mathbb{E}_{s\sim d^{\pi_\theta}_\mu}\mathbb{E}_{a\sim\pi_\theta(\cdot|s)}\Big[Q^{\pi_\theta}(s,a)\nabla_\theta \log \pi_\theta(a|s)\Big]. \tag{5}$$

(c) Let us check if the PG expressions used in some well-known papers are indeed correct.

- ACKTR, published in NeurIPS 2017 (available at `https://arxiv.org/abs/1708.05144`), is one variant

of the policy gradient algorithm. The paper mentioned a PG expression in Section 2.1. Is this PG expression correct? Please justify your answer and make the correction if you think their expression is wrong.

- ACER, published in ICLR 2017 (available at https://arxiv.org/abs/1611.01224) is yet another well-known variant of PG. The paper mentioned their PG expression of interest in Equation (1) in Section 2. Is this PG expression correct? Please justify your answer and make the correction if you think their expression is wrong.

Problem 3 (Baseline for Variance Reduction) (7+7+6=20 points)

Consider an example similar to that in the slides of Lecture 6 for explaining the baseline. Suppose that there are only 1 non-terminal starting state (denoted by s) and 3 actions (denoted by a, b, c) in the MDP of interest. After applying any of the actions in the starting state s, the MDP would evolve from s to the terminal state, with probability 1. Moreover, consider the following setting:

- The rewards are deterministic and the reward function is defined as $r(s, a) = 100$, $r(s, b) = 99$, and $r(s, c) = 98$. Moreover, there is no terminal reward.
- Let us consider a softmax policy with parameters $\theta_a, \theta_b, \theta_c$ such that $\pi_\theta(\cdot|s) = \exp(\theta_\cdot)/(\exp(\theta_a)+\exp(\theta_b)+\exp(\theta_c))$. Moreover, the parameters are currently $\theta_a = 0$, $\theta_b = \ln 3$, and $\theta_c = \ln 2$.
- We would like to combine PG with SGD. At each policy update, we would construct an unbiased estimate $\widehat{\nabla} V$ of the true policy gradient $\nabla_\theta V^{\pi_\theta}$ by sampling one trajectory (Note: $\widehat{\nabla} V$ is a random vector. In this example, each trajectory has only one time step, and $s_0 = s$, a_0 is either a, b, or c, and s_1 is the terminal state).

(a) What are the mean vector of $\widehat{\nabla} V$ (denoted by $\mathbb{E}[\widehat{\nabla} V]$) and the covariance matrix of $\widehat{\nabla} V$ (i.e., $\mathbb{E}[(\widehat{\nabla} V - \mathbb{E}[\widehat{\nabla} V])(\widehat{\nabla} V - \mathbb{E}[\widehat{\nabla} V])^\intercal]$)?

(b) Suppose we leverage the value function $V^{\pi_\theta}(s)$ as the baseline and denote by $\tilde{\nabla} V$ the corresponding estimated policy gradient. Then, what are the mean vector and the covariance matrix of $\tilde{\nabla} V$? (Note: $\tilde{\nabla} V$ is also a random vector)

(c) Let $B(s)$ denote a baseline function and ∇V_B be the corresponding estimated policy gradient (∇V_B is again a random vector). Suppose we say that a baseline function $B(s)$ is *optimal* if it attains the minimum trace of the corresponding covariance matrix of ∇V_B among all possible state-dependent baselines. Please try to find one such optimal $B(s)$.

Problem 4 (Policy Gradient Algorithms With Function Approximation) (15+15+15=45 points)

In this problem, we will implement three policy gradient algorithms with the help of neural function approximators: (1) Vanilla REINFORCE, (2) REINFORCE with value function as the baseline, and (3) REINFORCE with Generalized Advantage Estimation (GAE). For the pseudo code of the algorithms, please see the slides of Lectures 6-7. You may write your code in either PyTorch or TensorFlow (though the sample code presumes PyTorch framework). Moreover, you are recommended to use the package *Weight & Biases* to keep track of the loss terms and other related quantities of your implementation. If you are a beginner in learning the deep learning framework, please refer to the following tutorials:

- PyTorch: https://pytorch.org/tutorials/
- Weight and Biases: https://docs.wandb.ai/tutorials

For the deliverables, please submit the following:

- Technical report: Please summarize all your experimental results in 1 single report (and please be brief)
- All your source code

- Your well-trained models (REINFORCE with a baseline, without a baseline, and with GAE) saved in either .pth files or .ckpt files

(a) We start by solving the simple "CartPole-v1" problem (https://gymnasium.farama.org/environments/classic_control/cart_pole/) using the vanilla REINFORCE algorithm. Read through **reinforce.py** and then implement the member functions of the class **Policy** and the function **train**. Please briefly summarize your results in the report (including the snapshot of the Tensorboard record) and document all the hyperparameters (e.g. learning rates and NN architecture) of your experiments.

(b) Based on the code for (a), implement the REINFORCE algorithm with a baseline and solve the "LunarLander-v3" problem, which has slightly higher state and action space dimensionality (https://gymnasium.farama.org/environments/box2d/lunar_lander/). Note that you are allowed to **design a baseline function on your own** (e.g., the baseline can either be a handcrafted state-dependent function, the value function, or some function learned directly from the trajectories). Please clearly explain your choice of the baseline function in the report. Save your code in another file named **reinforce_baseline.py**. Please add comments to your code whenever needed for better readability. Again, please briefly summarize your results in the report (including the snapshot of the Tensorboard record) and document all the hyperparameters of your experiments. (Note: As LunarLander is a slightly more challenging environment than CartPole, it might require some efforts to tune the hyperparameters, e.g., learning rates or learning rate scheduler. You could either do grid search or even use some more advanced techniques such as the evolutionary methods. Typically, the LunarLander problem is considered solved if the testing episodic return is above 200)

(c) Based on the code for (a), implement the REINFORCE algorithm with the Generalized Advantage Estimation (GAE) as your value prediction. Please run the code on "LunarLander-v3". Note that there is a hyperparameter λ in the GAE algorithm, **please try three different values of** λ and summarize your results of these three choices clearly in the report. For more details about GAE, please refer to the slides of Lectures 7-8 and the original paper (https://arxiv.org/abs/1506.02438). Please briefly explain your implementation of GAE in your report. Also, please save your code in another file named **reinforce_gae.py**. **Moreover, please record a short video (3-5 minute long) to describe your design and the experimental results that you observe.** Last but not least, please add comments to your code whenever needed for better readability.

Remark: Regarding the snapshot of the Weight & Biases records, please feel free to record any value that you aim to know. Here is an example:

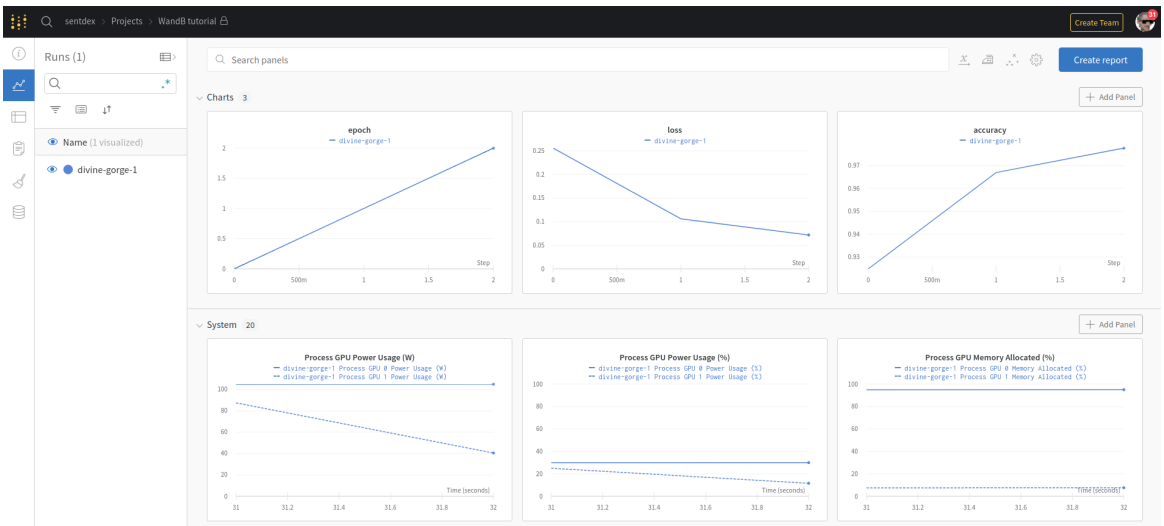

Figure 1: An example of the Weight & Biases record.